

**COMPUTATIONAL PORTFOLIO: PREDICTING RECENT
AIRBNB LISTING ACTIVITY AND DISCOVERING LISTING
STRUCTURES USING SUPERVISED AND UNSUPERVISED
LEARNING TECHNIQUES**

Table of Contents

1. Introduction.....	4
2. Data Understanding and Preparation.....	4
3. Focused Exploratory Data Analysis.....	7
3.1 Room-type imbalance.....	7
3.2 Activity by room type.....	8
3.3 Neighbourhood patterns.....	10
3.4 Numeric distributions.....	11
4. Supervised Learning.....	11
4.1 Problem definition.....	11
4.2 Tree-based model: Random Forest.....	12
4.3 Naive Bayes.....	16
4.4 Feed-forward neural network.....	18
5. Exploratory Structure Discovery.....	21
5.1 Choice of clustering technique.....	22
5.2 Variables selected.....	22
5.3 Preprocessing.....	23
5.4 Parameter selection.....	24
5.5 Cluster interpretation.....	24
5.6 Relation to supervised learning.....	25
6. Evaluation and Comparison.....	26
6.1 Evaluation metrics.....	26
6.2 Supervised model comparison.....	27
6.3 Clustering evaluation.....	27
6.4 Summary of modelling techniques.....	28
6.5 Broader comparison.....	28

7. Reflection and Conclusions.....	29
Reference:.....	31
Appendix.....	34

1. Introduction

The analysis was conducted using the Paris Airbnb Listings dataset, which has 81,853 observations and 18 features about listing and host details. These features contain information about the host, neighborhoods, type of room, review activity, minimum stay requirement, and yearly availability. However, two features that could have been used for an ordinary price prediction for an Airbnb, namely price and neighborhood_group, were completely missing from the dataset. As such, they were unusable, and the analysis was shifted to another supervised-learning goal.

The goal of supervised learning is to decide if a listing has gotten at least one review within the past 12 months. The target binary variable is defined as

$$active_{listing} = \begin{cases} 1, & \text{if } number_of_reviews_ltm > 0 \\ 0, & \text{if } number_of_reviews_ltm = 0 \end{cases}$$

The significance of the problem lies in the fact that recent review activity serves as an observable measure of recent listing activity (Gao Jr, (2025)). While reviews do not equal bookings, occupancy, or income generation, it is still a tangible measure of whether the listing was active in the past year.

In order to examine the problem, the portfolio utilizes Random Forest, Gaussian Naive Bayes, and a feedforward neural network as supervised classification algorithms, and uses K-means clustering as an unsupervised algorithm for structure learning and PCA for visualization of clusters.

2. Data Understanding and Preparation

The dataset contained 81,853 observations and 18 variables, with each observation representing an Airbnb listing. At first, it was checked that there are no duplicate rows and duplicate listing IDs in the dataset, which means that all records are unique. It is possible to say that the data include 20 neighbourhoods and 4 different room types, which adds spatial and property diversity for further research.

Based on the type of measurement, the variables can be categorized into groups. The variables id and host_id act as identifiers and were not used in modeling as predictors. The

neighbourhood and room_type are categorical predictors. As numerical predictors, minimum_nights, number_of_reviews, and availability_365 are used. last_review variable is considered a date predictor, whereas name, host_name, and license are considered as string predictors.

	data_type	missing_count	missing_percentage	unique_values
price	float64	81853	100.000	1
neighbourhood_group	float64	81853	100.000	1
last_review	object	17960	21.942	3513
reviews_per_month	float64	17960	21.942	889
license	object	17256	21.082	52487
host_name	object	20	0.024	11297
name	object	0	0.000	76096
id	int64	0	0.000	81853
neighbourhood	object	0	0.000	20
host_id	int64	0	0.000	59587
room_type	object	0	0.000	4
longitude	float64	0	0.000	44767
latitude	float64	0	0.000	38993
number_of_reviews	int64	0	0.000	629
minimum_nights	int64	0	0.000	108
calculated_host_listings_count	int64	0	0.000	95
availability_365	int64	0	0.000	366
number_of_reviews_ltm	int64	0	0.000	142

Duplicate rows: 0
 Duplicate listing IDs: 0
 Entirely missing columns: ['neighbourhood_group', 'price']

Figure 1. Percentage of missing values across variables in the Paris Airbnb dataset.

(Source: Self-created)

In the course of performing a data quality audit, some problems with missing values were revealed. The price and neighbourhood_group contain 81,853 missing values each, which is 100.00% of the observations. Such variables could not be used for further analysis; hence, they were deleted. There were found 17,960 missing values (21.94%) in last_review and reviews_per_month; the same missingness ratio is expected to be structurally induced, as listings which do not have any reviews cannot have a review date and the number of monthly

reviews (Vasudevan (2025)). License also demonstrates high missingness with 17,256 (21.08%) missing values, and it could not be considered as a predictor due to its high cardinality as a string predictor that reflects regulatory status rather than behaviour of the listing. Though host_name has 20 missing values (0.02%), it was not chosen as a predictor since personal names have no analytic meaning for the given model and will add cardinality.

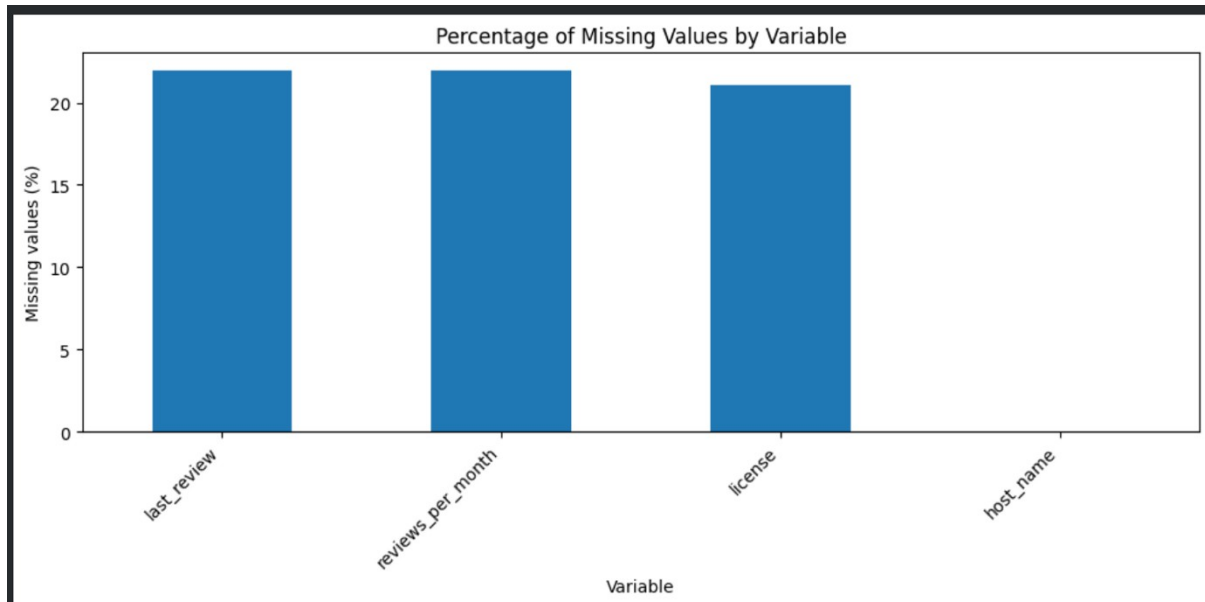


Figure 2: Percentage of missing values by Variables

(Source: Self-created)

For performing supervised learning, a target variable was generated based on number_of_reviews_ltm: listings having at least one review in the last 12 months are active ($y = 1$), and listings having no recent reviews are inactive ($y = 0$). This resulted in 42,978 active listings (52.51%) and 38,875 inactive listings (47.49%). Thus, the target class distribution is balanced, and there are fewer risks of majority class bias when training models.

As predictors, variables with potential explanations were chosen in order to avoid target leakage. The following variables were chosen as predictors: neighbourhood, room_type, latitude, longitude, minimum_nights, number_of_reviews, calculated_host_listings_count, availability_365. Leakage controls were performed carefully. number_of_reviews_ltm was deleted because it defines the target variable. In the same way, last_review and reviews_per_month were deleted because these variables show recent review activity.

Afterwards, several preprocessing procedures were performed. Categorical predictors were transformed through one-hot encoding, and missing numerical values were imputed by median imputation and missing categorical values by mode imputation. The data were split into training and test datasets through 80:20 stratified splitting, so that there are 65,482 training and 16,371 test observations, while maintaining the proportion of classes in the target. The feature scaling was performed for Gaussian Naïve Bayes, the feed-forward neural network, and K-means clustering, because they are sensitive to predictor magnitudes. Random Forest does not require feature scaling, since tree-based algorithms are invariant to monotonous transformation of predictors' scale.

3. Focused Exploratory Data Analysis

This type of analysis was limited to those aspects that would have an impact on the models used later. These include imbalance in the data categories, differences in listings based on room type and neighborhood, and distribution features of numeric variables.

3.1 Room-type imbalance

An interesting imbalance exists in the variable room type. The total number of entire homes/apartments was 72,297, whereas private rooms were 8,727, hotel rooms amounted to 664, and shared rooms had 165 observations. Mathematically speaking, $n(\text{Entire home/apt}) \gg n(\text{Private room}) \gg n(\text{Hotel room}) > n(\text{Shared room})$. It means that the entire homes/apartments prevail in the sample, and the shared rooms constitute a tiny category.

There are two consequences from this observation. Room type seems to be a valuable predictor because it refers to different market segments. In addition, the estimates for rare categories, especially for shared rooms, have higher variance and thus must be treated carefully. Any effect observed for shared rooms will be highly unstable due to the small number of observations.

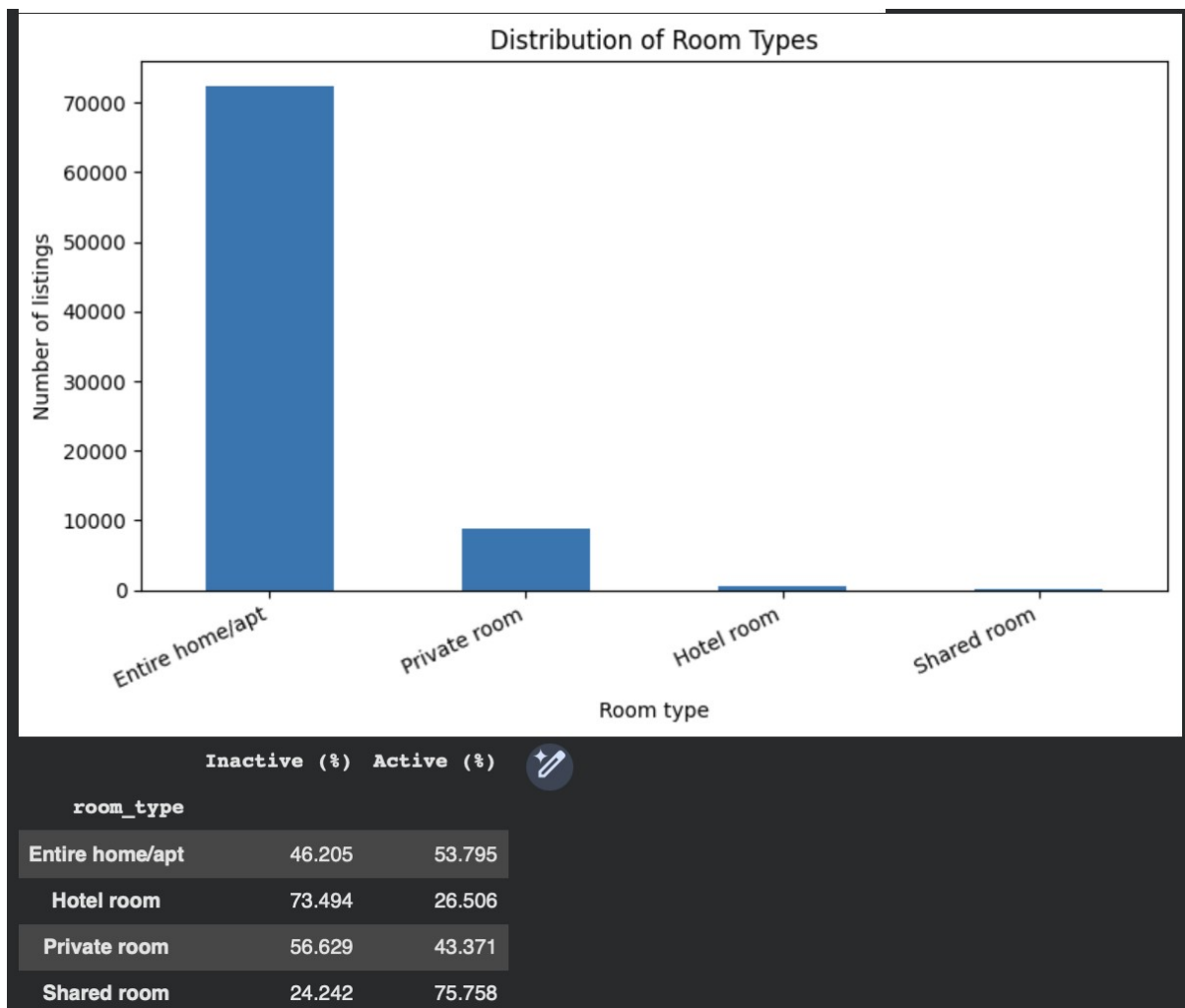


Figure 3: Distribution of Room Types

(Source: Self-created)

3.2 Activity by room type

Listing activities varied based on room type. The overall home/apt had an activity listing of 53.80%, while the private room had 43.37%. Activity listing for hotel rooms and shared rooms was 26.51% and 75.76%, respectively (Costa, (2025)). Based on these results, one could argue that room type has some predictive power for the dependent variable.

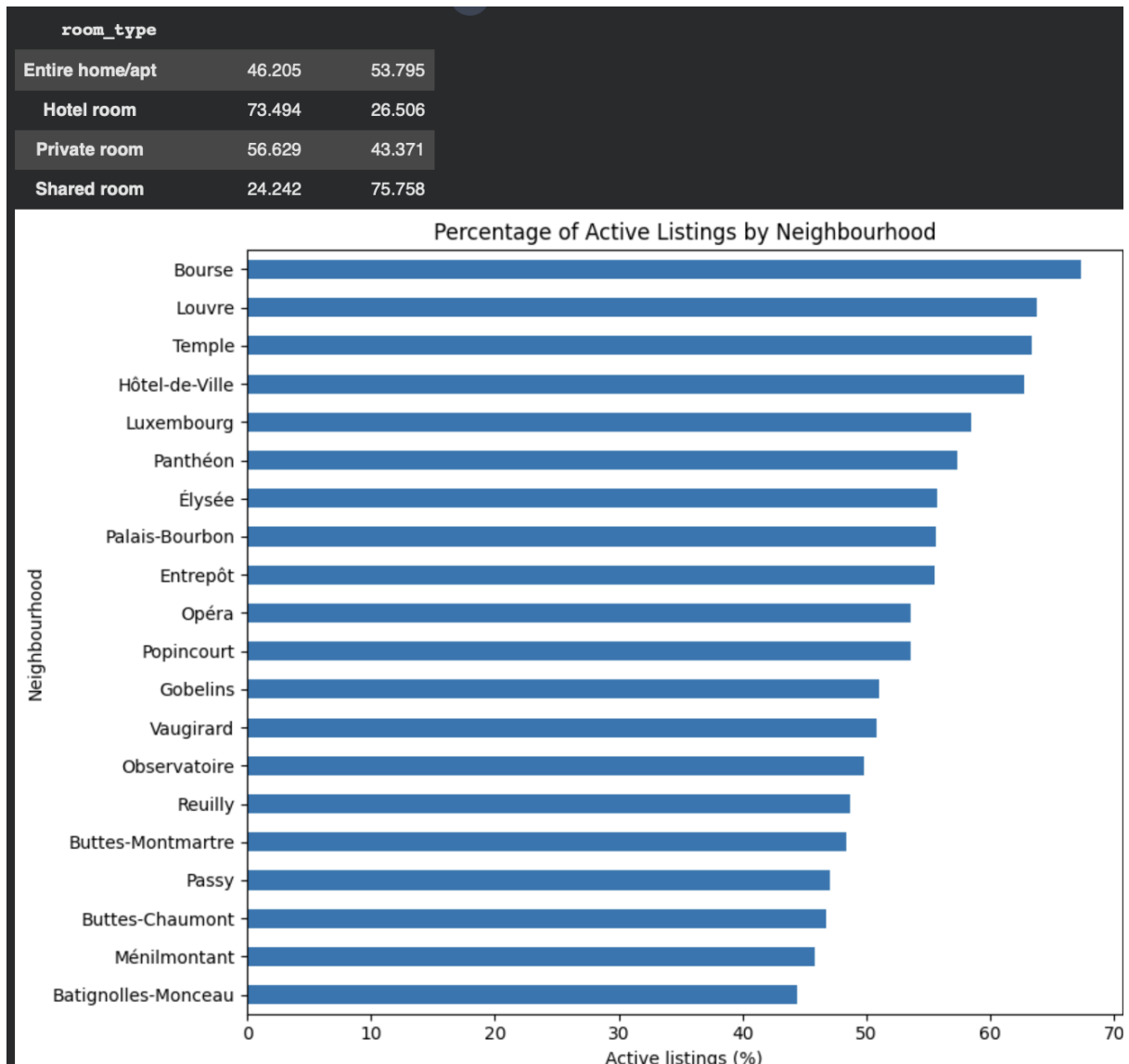


Figure 4: Percentage of Active Listings by Neighbourhood

(Source: Self-created)

However, one needs to be careful about the high percentage of activity listing in shared rooms. Although having the highest percentage, it has been generated from just 165 listings. In case of a small sample size, the

$\hat{p} = \frac{\text{number of active listings}}{\text{total listings} \in \text{the group}}$ is more sensitive to small changes in the numerator and denominator. Accordingly, the shared-room result was treated as indicative rather than conclusive.

3.3 Neighbourhood patterns

The results from the neighbourhood-level analysis highlighted scale and activity disparities among various regions. The largest neighbourhoods included Buttes-Montmartre (with 8,685 properties), Popincourt (with 7,204), and Vaugirard (with 6,633). On the other hand, the smallest neighbourhoods were Louvre (with 1,858). Similarly, the activity percentages varied significantly from one neighbourhood to another. For instance, the highest activity percentages were recorded for Bourse (67.3%), Louvre (63.7%), Temple (63.4%), and Hôtel-de-Ville (62.7%). Conversely, the lowest percentages occurred for Batignolles-Monceau (44.4%), Ménilmontant (45.9%), and Buttes-Chaumont (46.7%).

neighbourhood		
Batignolles-Monceau	0.444	5557
Ménilmontant	0.459	4438
Buttes-Chaumont	0.467	4481
Passy	0.471	5147
Buttes-Montmartre	0.483	8685
Reuilly	0.487	3321
Observatoire	0.498	3027
Vaugirard	0.508	6633
Gobelins	0.510	2659
Popincourt	0.535	7204
Opéra	0.536	3987
Entrepôt	0.555	5584
Palais-Bourbon	0.556	2400
Élysée	0.557	2625
Panthéon	0.573	2754
Luxembourg	0.584	2476
Hôtel-de-Ville	0.627	2579
Temple	0.634	3549
Louvre	0.637	1858
Bourse	0.673	2889

Figure 5: Neighbourhood

(Source: Self-created)

Clearly, the above patterns demonstrated that listing activities occur unevenly. In other words, there were differences among administrative regions that were influenced by variations in tourism demand and local market structures.

3.4 Numeric distributions

Several numeric variables showed considerable right-skewness with long upper tails. Median minimum nights were 3, while the maximum was 1,000. Total reviews had a median of 7 and the maximum was 4,040. Host listings count was median 1 but max 816. This means that hosts have very few listings on average, while a few hosts operate on a huge scale. Finally, availability had a median of 74 days while the maximum was 365 days.

This information indicates heavy asymmetry in the data with some high-value outliers. Since the plots would be crowded with such extreme observations, the exploratory analysis plots were only drawn up to the 99th percentile to understand the structure of the data without being crowded by outliers (Camatti et al., (2024)). In case of clustering, the selected variables were truncated to the 99th percentile because the distance-based techniques like the K-means algorithm are particularly affected by extreme observations.

Generally, the exploratory analysis indicated non-linear and skewed relationships in the data along with interaction. This makes the use of Random Forest and a feedforward neural network as good predictive models. Gaussian Naïve Bayes model was kept for comparative purposes.

4. Supervised Learning

4.1 Problem definition

Formulation of the problem under supervised learning took place through classifying it as a binary classification problem where the aim was to predict whether the listing on Airbnb had been used recently. Defining the target variable was done through looking at the review activity during the past 12 months, whereby:

$$y_i =$$

$$\in \{cases\}$$

$$1, \& \begin{cases} \text{if listing } i \text{ received at least one review} \in \text{the last 12 months} \end{cases} \\ 0, \& \begin{cases} \text{otherwise} \end{cases} \\ \begin{cases} \text{cases} \end{cases}$$

In that light, the main assumption in the analysis is the fact that a listing with at least one review in the past 12 months is considered currently active. The target in question is realistic since the existence of the recent review proves that there were at least some transactions in the time frame in question. The availability of reviews in platform data is among the few signs that can prove recent activity, since there is no other data on bookings/revenue.

The class distribution was slightly imbalanced but not dramatically so, and the current listings were the majority class. Thus, it was possible to use classification techniques without resorting to specialised methods for imbalanced classes, though class balancing would still be helpful in the case of class-sensitive models. Generally speaking, the current target reflects recent activity on the platform rather than commercial success.

This way of constructing the target has several shortcomings. First of all, it is possible that there were transactions without reviews, and one review does not prove any specific level of activity or revenues (Amarasinghe Arachchige (2023)). The guest-host interaction is different for different listings, so the lack of reviews does not necessarily reflect inactivity. Similarly, one review cannot tell whether a listing had many bookings or few.

4.2 Tree-based model: Random Forest

The chosen algorithm is a Random Forest classifier, an ensemble technique that is good at dealing with tabular data and capturing complex nonlinear relationships between features and the target variable. While a single decision tree is unstable and sensitive to fluctuations in the training data, a Random Forest averages the predictions of multiple trees built on bootstrapped datasets and different combinations of predictors. This approach decreases the variance and tends to increase generalization performance. It also takes into account interaction effects that are important in this case since listing activity depends on interactions between the geographic location, room type, review scores and host attributes. After encoding categorical variables, the model will be able to handle mixed types of predictors. Furthermore, it generates feature importances, which can be used for interpretation although do not have any causal meaning.

This choice agrees with the lecture where tree-based ensembles were presented as a way to overcome the drawbacks of individual trees.

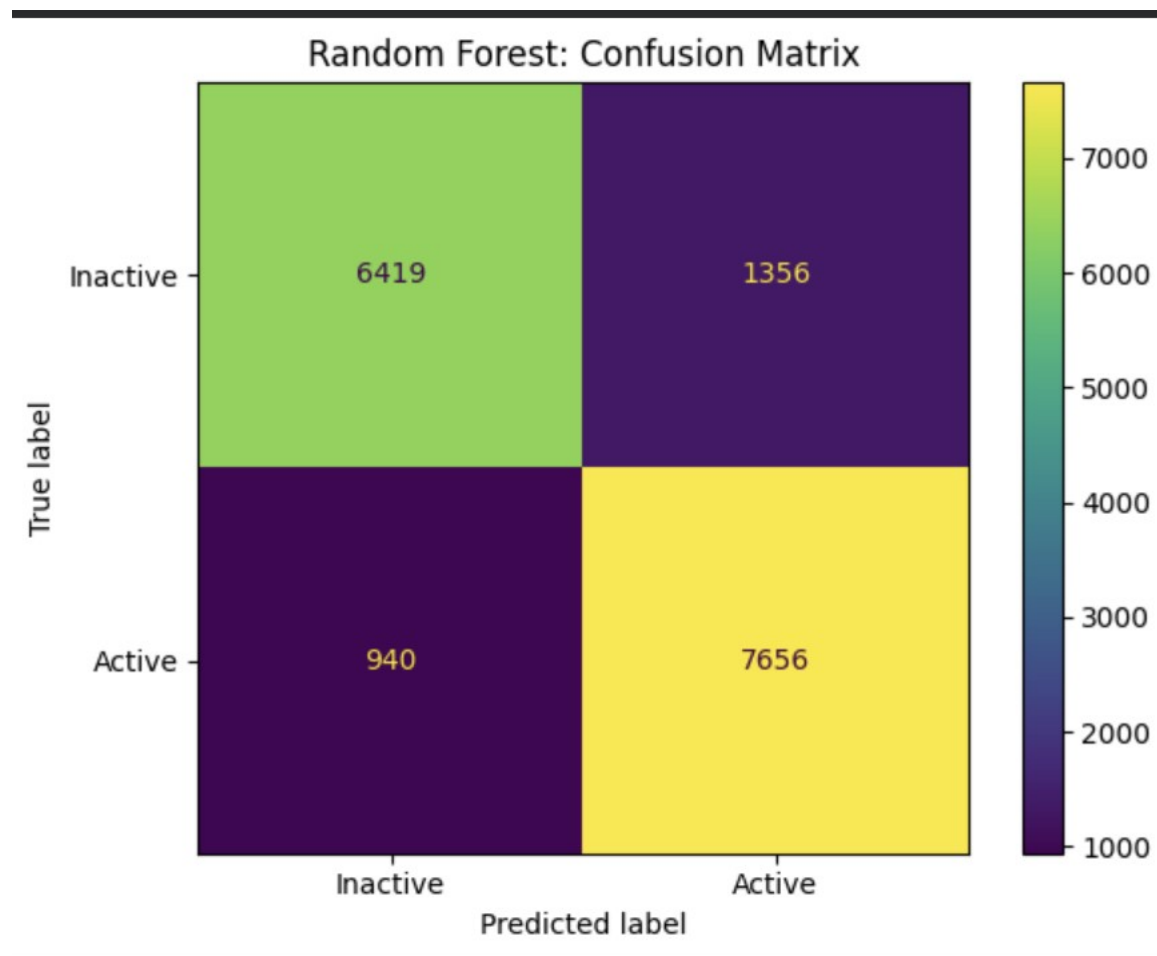


Figure 6: Random Forest: Confusion Matrix

(Source: Self-created)

As for the preprocessing stage, I tried to keep it simple. Numerical variables were imputed with the median value; categorical variables were one-hot encoded. Feature scaling was unnecessary since tree-based models use thresholding and are not impacted by the scale of predictors. Final parameter choices are:

- (n_estimators = 250)
- (max_depth = 16)
- (min_samples_leaf = 5)
- (class_weight = balanced)
- (random_state = 42)

In addition, these parameters were moderate enough for regularisation, which reduced overfitting but required minimal fine-tuning of hyperparameters. Specifically, imposing a limit on tree depth and the minimum number of observations at terminal nodes limited the possibility of partitioning into too specific segments.

The Random Forest demonstrated the best prediction performance compared to other supervised learning methods. For the test set, it yielded the following metrics: 0.860 accuracy, 0.850 precision, 0.891 recall, 0.870 F1-score and 0.939 ROC-AUC, with fit time being 19.024 seconds. Such performance speaks well about ranking capabilities and a good balance of precision and recall measures.

Importance estimates demonstrated that the Random Forest model was driven by historical data regarding reviews. The most important predictors were total reviews (0.506), availability (0.239) and minimum nights (0.144). Host listing count (0.042), latitude (0.023) and longitude (0.021) also had some contribution to the predictions, although much lower than that of historical data.

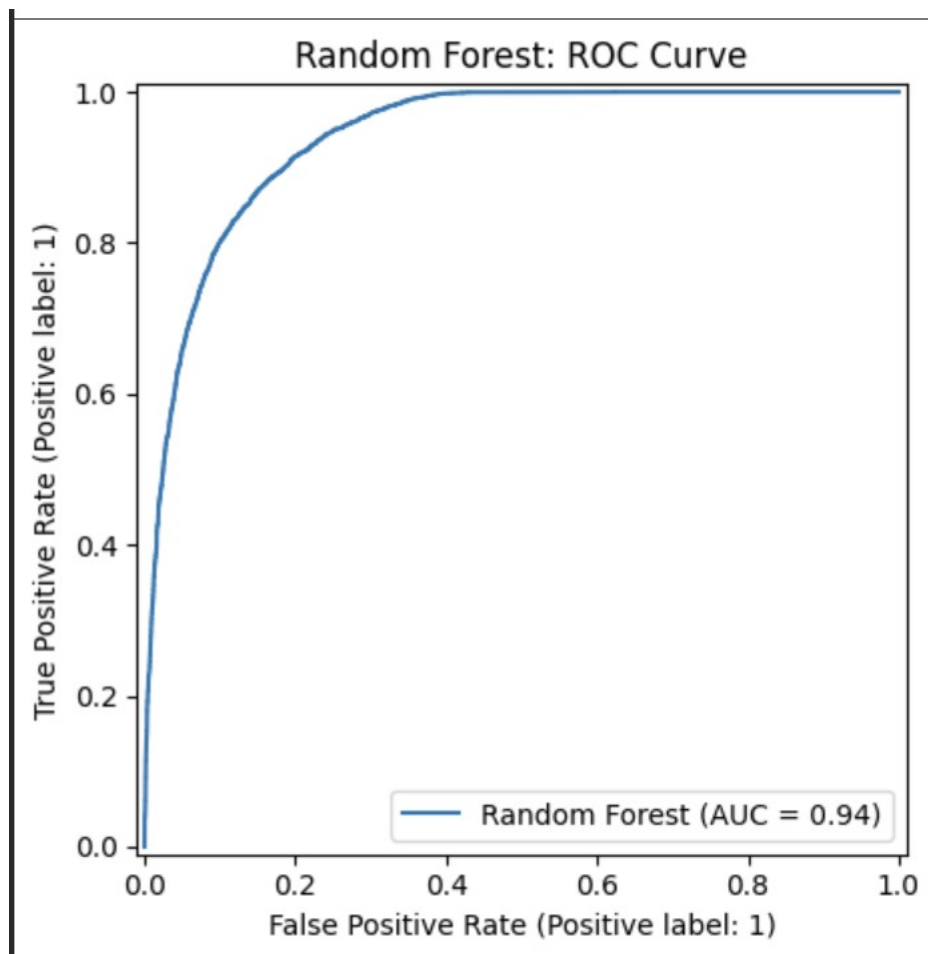


Figure 7: Random Forest: ROC Curve

(Source: Self-created)

First of all, the Random Forest model possesses three key advantages: the highest quality of predictions, good modelling of non-linear relations and interactions, as well as good interpretability in terms of feature importance. At the same time, the limitations of the model should also be mentioned. First of all, impurity-based feature importance measures are descriptive rather than causal; hence, they do not imply that some feature drives activity in a causal way.

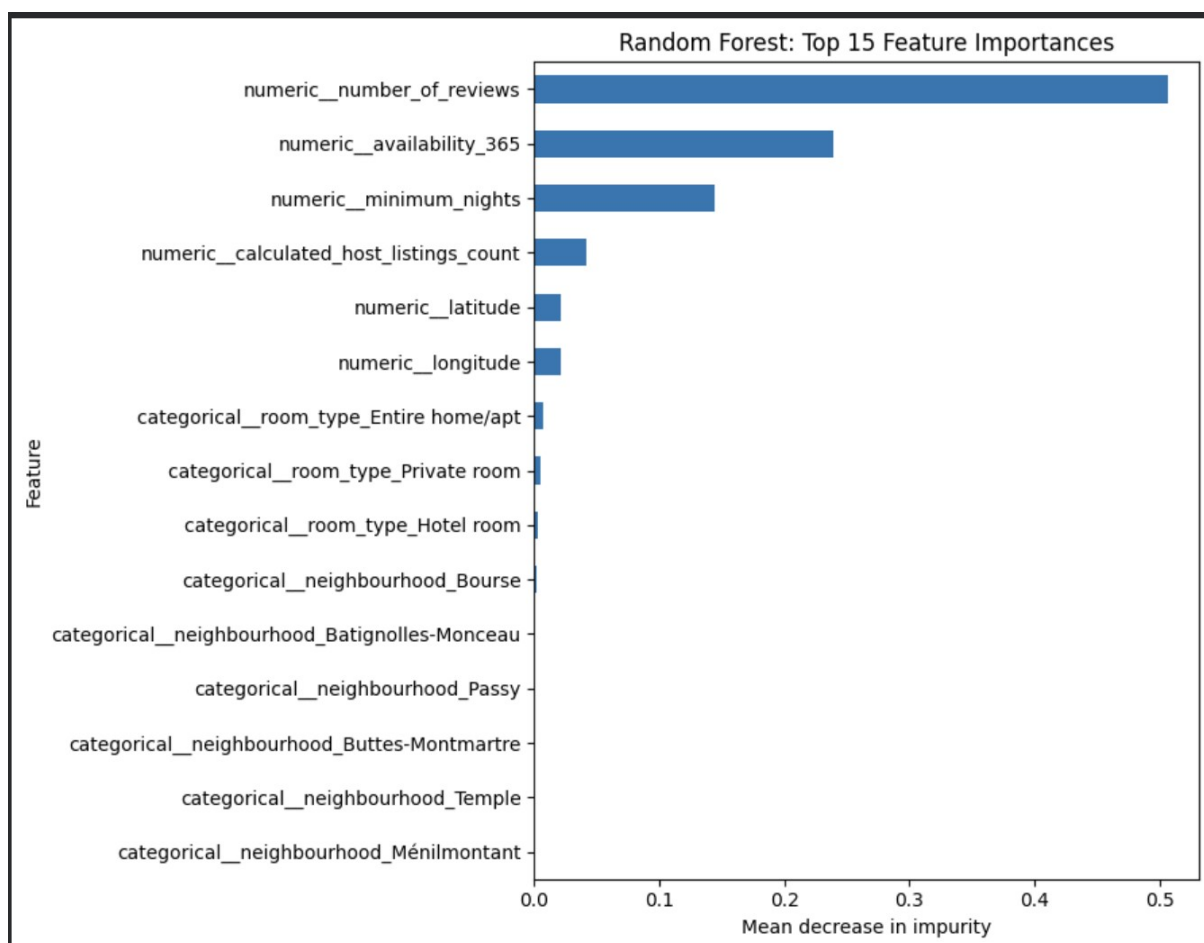


Figure 8: Random Forest: Top 15 Feature Importances

(Source: Self-created)

4.3 Naïve Bayes

Gaussian Naive Bayes was used as an easy-to-understand baseline algorithm based on probability theory with Bayes' theorem. With a feature vector $x = (x_1, x_p)$, the algorithm assigns to class $y \in \{0,1\}$ using:

$$x = (x_1, \dots, x_p)$$

The algorithm is described as "naive" due to its assumption of feature independence given the class label. In the case of the Gaussian classifier, additional assumptions state that continuous features have approximately normally distributed class-conditional distributions.

None of these assumptions can be expected to hold in the case of Airbnb data. First, reviews, availability, and minimum nights likely influence each other through similar market forces. Second, some numerical features are heavily skewed, violating the assumptions about their Gaussian class-conditional distributions (Romanillos & Moya-Gómez (2023)). Third, geographic factors could interact with the types of rooms and host characteristics rather than being conditionally independent. Despite this, the Naïve Bayes approach remains helpful as a reference model, as it is fast, straightforward to implement, and often works surprisingly well with restrictive assumptions.

Numerical features were standardised, categorical features were one-hot encoded, and all missing data points were imputed before fitting the model. Standardisation was performed since the Gaussian Naïve Bayes models class-conditional mean and variance of each feature and therefore requires consistent feature scaling.

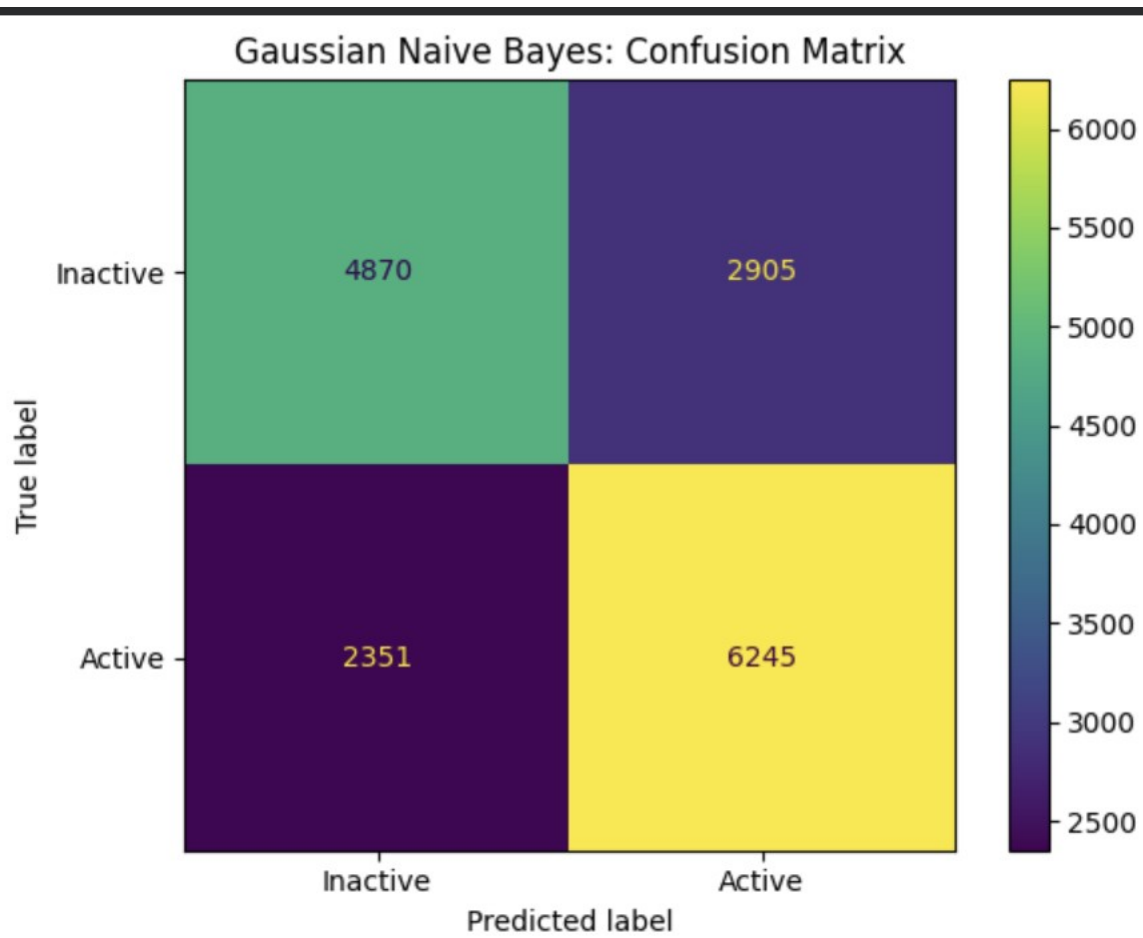


Figure 9: Gaussian naïve Bayes: Confusion Matrix

(Source: Self-created)

The model reached the following metrics: 0.679 accuracy, 0.683 precision, 0.727 recall, 0.704 F1 score, and 0.752 ROC-AUC, having taken just 0.181 seconds to fit. This was the quickest supervised learning model tested. However, its prediction performance turned out to be very weak compared to Random Forest and the neural network.

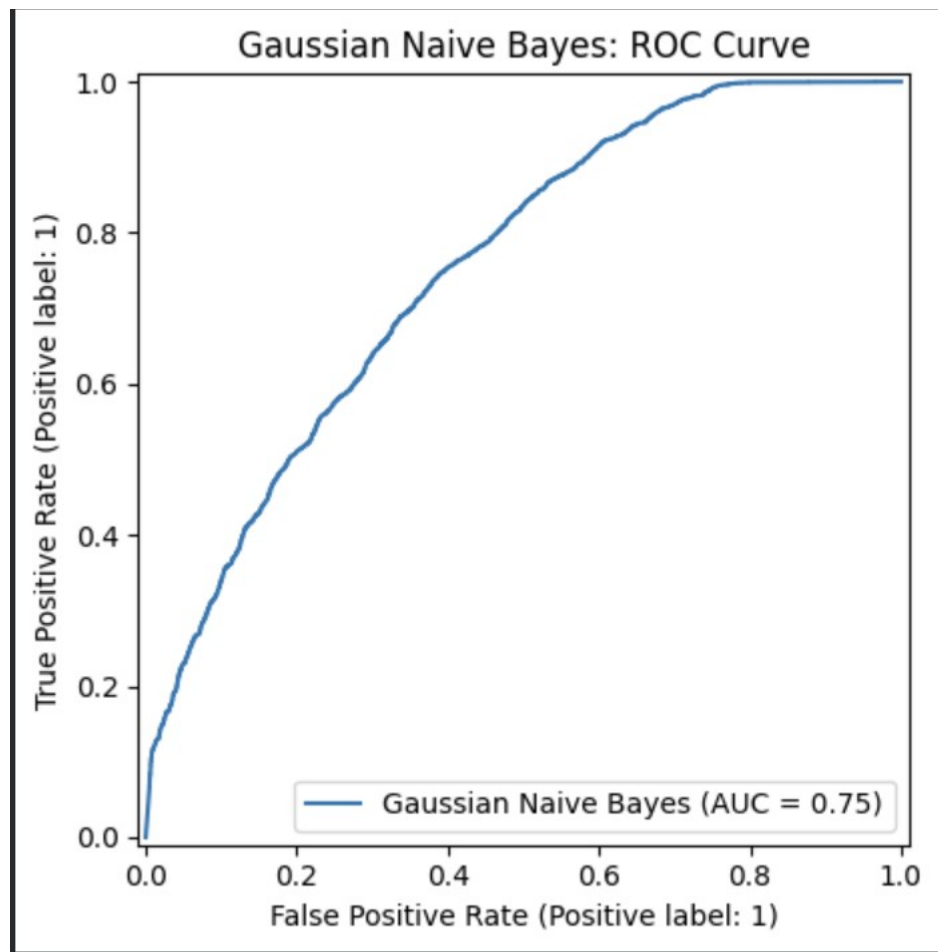


Figure 10: Gaussian Naïve Bayes: ROC Curve

(Source: Self-created)

On the positive side, it is incredibly fast, conceptually simple, and produces probabilistic outputs, serving as a good reference point for performance comparison. The weakness of the model stemmed from its assumptions: the conditional independence assumption does not hold true, it cannot account for non-linear interactions, and its prediction performance was significantly lower than that of competing algorithms.

4.4 Feed-forward neural network

The performance of a flexible nonlinear learner relative to classical machine learning models was tested using a feed-forward neural network. The network structure is consistent with the discussion of perceptron architecture, hidden layers, feedforward, activation, and back-propagation in the lecture. Every node in the network performs the computation of a weighted

input sum and applies a non-linear function to it. For hidden layer nodes, the rectified linear unit function was used:

$$\mathcal{L}\{ReLU\}(z) = \mathcal{L}(0, z)$$

For the output layer, a sigmoid activation transformed the final score into a probability:

$$\mathcal{L}(z) = \mathcal{L}\{1\}\{1 + e^{-z}\}$$

Once data pre-processing was done, numerical attributes were standardized, categorical attributes were one hot-encoded, and finally, the number of input attributes became 30 after encoding.

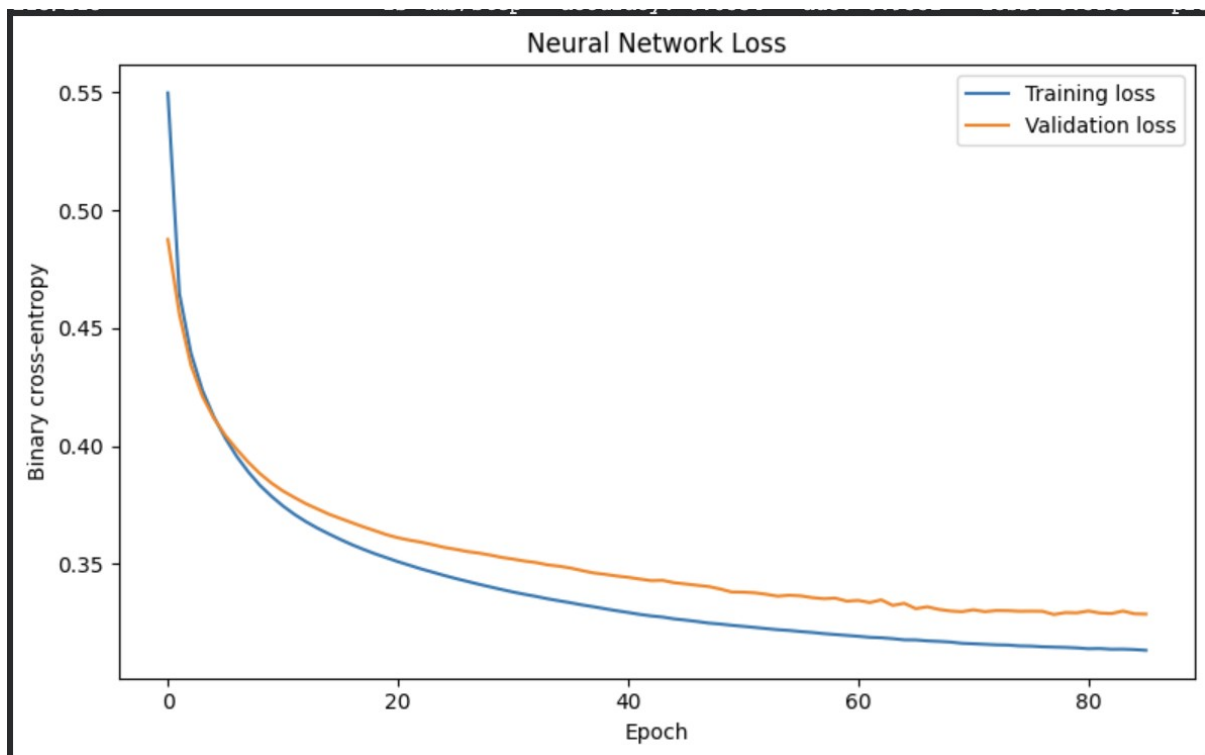


Figure 11: Neural Network Loss

(Source: Self-created)

In the final model architecture, there is an input layer with 30 features; the first hidden layer has 32 neurons and uses ReLU activation function, while the second hidden layer has 16 neurons and uses ReLU activation function, and lastly, the output layer has only one neuron

and uses the sigmoid activation function. The total number of trainable parameters is 1,537. It has been kept intentionally small due to the structured nature of the dataset.

The training process included the binary cross-entropy loss function,

$$\mathcal{L} = -\frac{1}{n} \sum_{i=1}^n [y_i \log(\hat{p}_i) + (1 - y_i) \log(1 - \hat{p}_i)]$$

along with the Adam optimiser and a learning rate of 0.001. Batch size of 256, maximum number of epochs of 100, and 15% holdout from the training dataset for validation were used. Additionally, early stopping with a patience of 8 was implemented to restore the best validation weights after additional training stops improving the validation loss. A class weighting technique was also used to mitigate the effects of class imbalance.

Those design choices were reasonable given the task of binary classification. ReLU allows building nonlinear hidden representations without suffering from vanishing gradients due to saturation in hidden layers. Sigmoid output produces a legitimate probability in [0,1], binary cross-entropy works naturally with Bernoulli outcomes, and early stopping serves as a regularisation mechanism.

The neural network showed an accuracy of 0.845, precision of 0.825, recall of 0.894, F1-score of 0.858, ROC-AUC of 0.925 and fitting time of 95.769 seconds. The best metric of the neural network is recall, which is the best among the three supervised models.

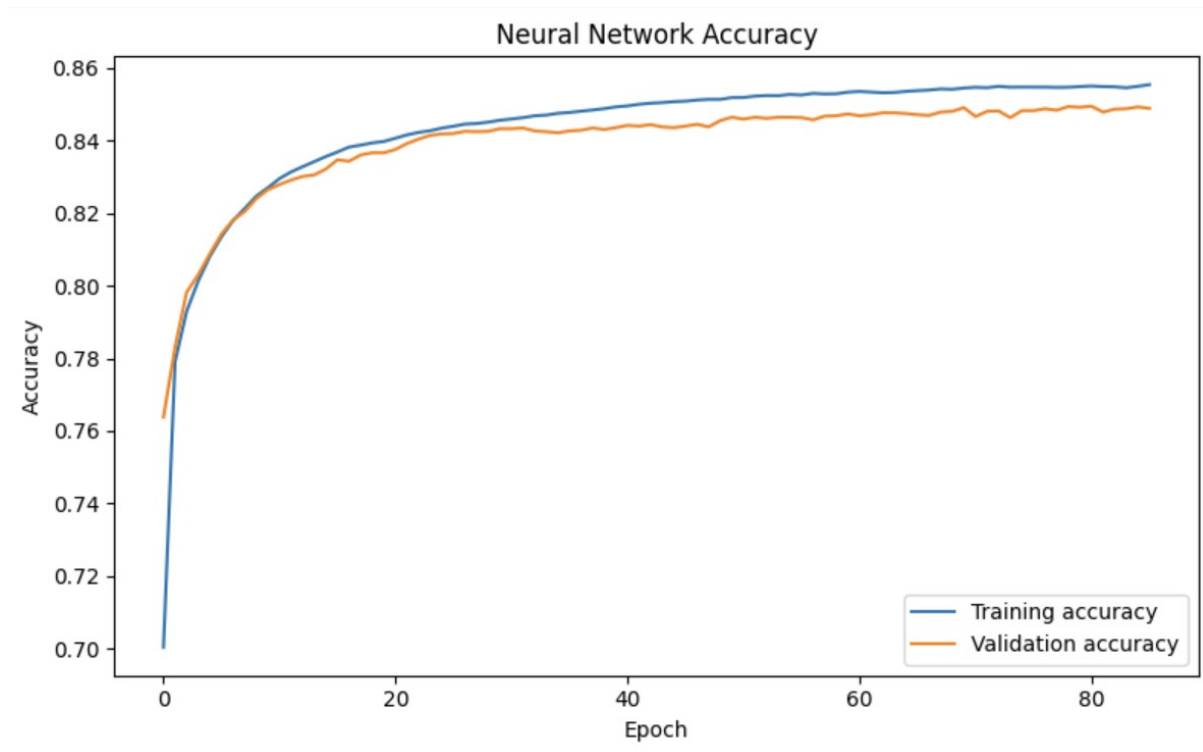


Figure 12: Neural Network Accuracy

(Source: Self-created)

The strengths of the model are its ability to model nonlinear interactions, flexibility and good recall (Motamedi et al., (2026)). The weaknesses of the model are obvious as well – its slow fitting process, lack of interpretability, and need for scaling and architectural decisions. Although the model is quite flexible, it did not show better results than Random Forest and therefore was not chosen as the supervised model.

5. Exploratory Structure Discovery

To supplement these supervised models, an unsupervised analysis involving clustering was conducted to ascertain whether there were any meaningful structural groupings within the Airbnb listings. It is important to note that the aim of this analysis was not predictive but rather to discover any meaningful structures within the listings.

5.1 Choice of clustering technique

K-means algorithm was chosen as the main unsupervised clustering method for four reasons. Firstly, it has been covered in great detail in the module and thus ensures consistency in

methodology that is required for this research. Secondly, the K-means algorithm generates centroids of each cluster, which means that the generated clusters can be profiled and interpreted quite easily. Thirdly, it is computationally efficient on a sample of 81,853 observations, which means that several solutions can be estimated without spending too much time on calculations (Alibakhshi & Hassannayebi (2025)). Lastly, the clusters can be characterized by median values of features and cross-tabulations.

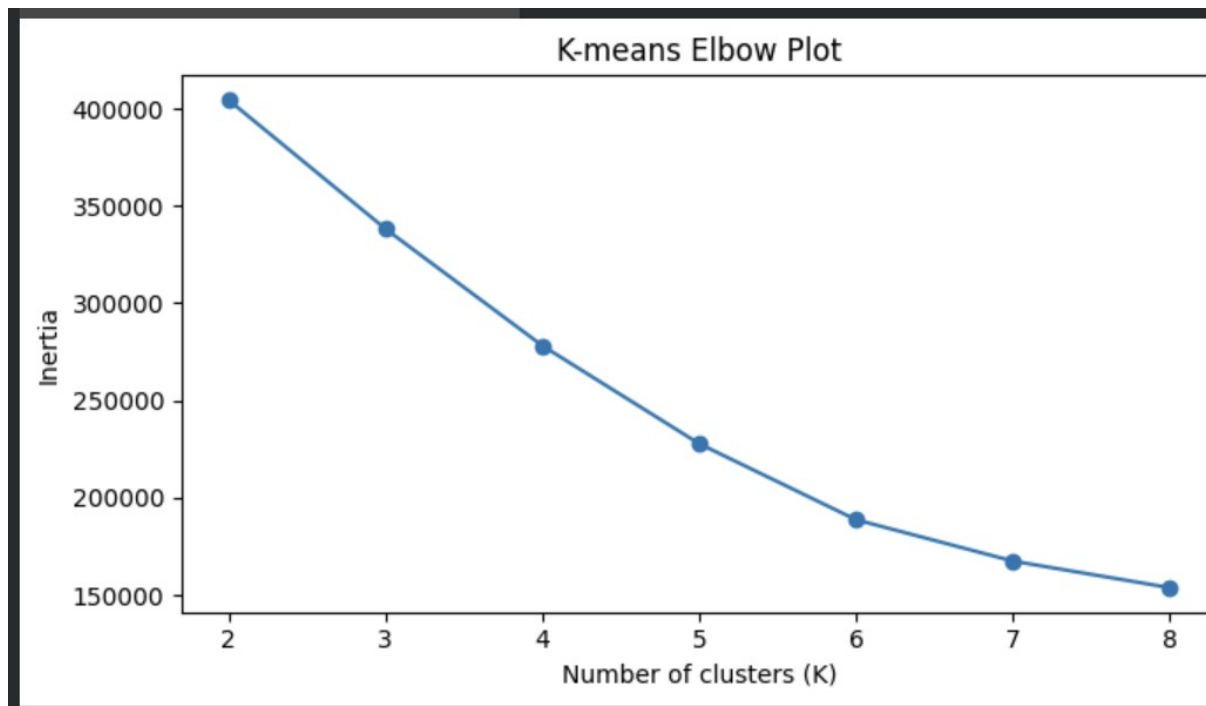


Figure 13: K-means Elbow Plot

(Source: Self-created)

5.2 Variables selected

The clustering input was limited to six numeric features: latitude, longitude, minimum nights, number of reviews, calculated host listings count, and availability over 365 days. Such an approach was chosen since these features cover several independent aspects of listing structure. Latitude and longitude define geographical position. Minimum nights define the stay restrictions set by the host. Number of reviews defines historical guest activities and transaction volume. Calculated host listings count is used as a measure of the host's scale in order to distinguish individual hosts from possibly bigger portfolio owners. Availability over 365 days is used as a measure of listing availability through the year.

The room type feature was not included in the clustering input due to K-means' reliance on Euclidean distance and its suitability for only continuous numeric variables. However, such a feature could be included after some transformation steps; otherwise, the results of clustering would have been distorted. Still, the room type feature was included for post hoc cluster interpretation purposes.

5.3 Preprocessing

Several preprocessing techniques were applied before clustering. The missing numeric values were imputed with the median since it is insensitive to distribution skewness, unlike the mean, which is very sensitive to extreme values. Skewed variables, especially minimum nights, number of reviews, and the calculated host listing count, were further trimmed to the 99th percentile. After imputation and trimming, all clustering variables were standardised to ensure that all the features were on the same scale.

The standardization was important since K-means minimizes the squared Euclidean distance within clusters,

$$\sum_{k=1}^k \sum_{i \in C_k} ||x_i - \mu_k||^2$$

(C_k) refers to cluster (k), and (μ_k) is the corresponding centroid. Without the standardisation step, variables that have greater numeric range would have more weight on the distance metric than those with smaller numeric range, leading to their undue influence in determining the clusters.

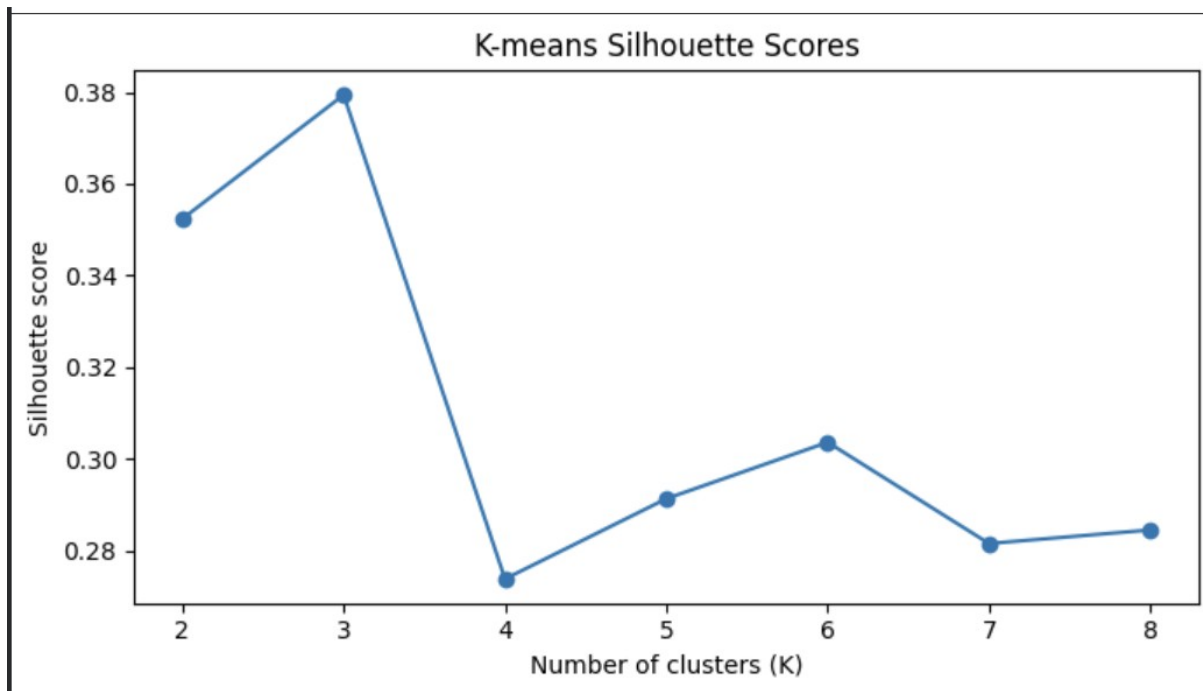


Figure 14: K-means Silhouette Scores

(Source: Self-created)

5.4 Parameter selection

The number of clusters used was selected by using silhouette analysis among various possible numbers of clusters ($K \in \{2, \dots, 8\}$). The silhouette scores for the different numbers of clusters were: 0.352 when ($K=2$), 0.379 when ($K=3$), 0.274 when ($K=4$), 0.291 when ($K=5$), 0.304 when ($K=6$), 0.281 when ($K=7$), and 0.284 when ($K=8$). The maximum value of the silhouette score occurred for ($K=3$), and this clustering model was also substantively interpretable. As a result, the three-cluster model was chosen.

5.5 Cluster interpretation

The third and final clustering solution identified three clearly differentiable listing segments. Cluster 0 was the largest segment, with 69,007 listings. The medians for this segment were 3 minimum nights, 9 reviews, 1 host listing, and 87 available days per year. Additionally, 58.57% of listings within this cluster were marked as active, and 86.64% of listings were entire homes/apartments. This segment can be considered as typical individual listings – mainly single-host listings with moderate availability and moderate past activity.

Cluster 1 included 4,373 listings and was mainly characterized by host scale. The medians of this segment were 3 minimum nights, 2 reviews, 210 host listings, and 231 available days. The active rate was 56.55%, and 94.76% of listings were entire homes/apartments. These results suggest professional-level hosting since listings correspond to very big portfolios and relatively high availability throughout the year (Chaudhary et al., (2024)). However, this cannot be taken as direct proof of commercial operation but rather a clear indication of such hosting activities.

Cluster 2 comprised 8,473 listings and stood out among other segments. The medians for this segment were 365 minimum nights, 1 review, 1 host listing, and 0 available days. Only 1.01% of listings from this cluster were active, while 98.73% of listings were entire homes/apartments. This cluster can be interpreted as listings with long minimum stays – listings with severe booking limitations, little recent activity, and zero availability.

5.6 Relation to supervised learning

These results also confirm our supervised learning results. Specifically, cluster three shows how those listings that have a very high requirement for minimum nights and low availability have extremely low review activity. It corresponds well to our classification algorithms, in which minimum nights and availability became the most important features predicting if the listing is recently active. Thus, this unsupervised method confirms once again that these features indeed carry some important structural information about the market.

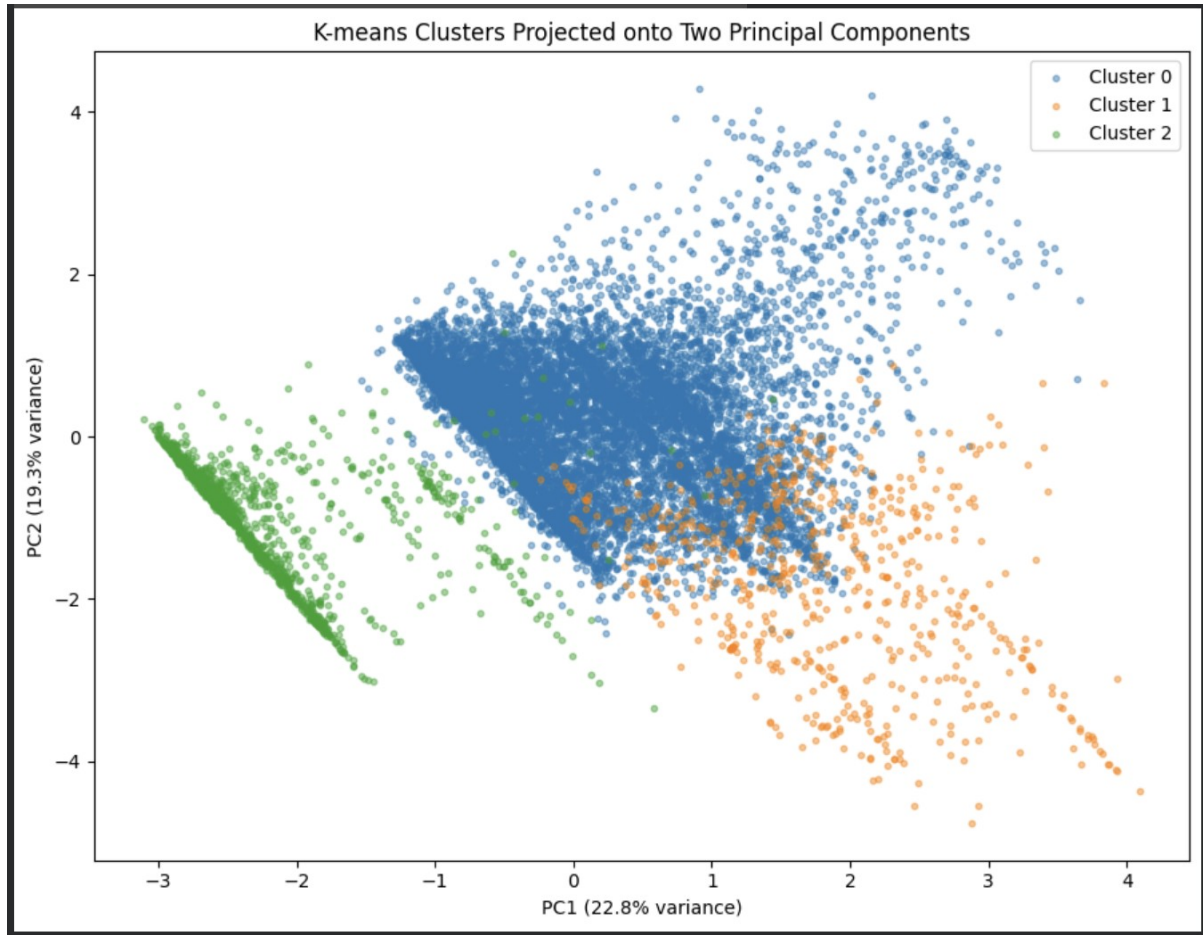


Figure 15: K-means Clusters Projected onto two principal components

(Source: Self-created)

As a final step, PCA was performed only to visualize the clustering result in two dimensions. Two leading components explain 42.13% of the variance (Puri et al., (2023, January)). Since the clustering was done in the six-dimensional space, the overlap on the two-dimensional plot does not indicate any flaws with our clustering result; on the contrary, it shows the loss of information that happens when a higher-dimensional structure is projected into just two dimensions.

6. Evaluation and Comparison

6.1 Evaluation metrics

In total, there were six measures used to assess the effectiveness of supervised models. One of those measures is Accuracy, which is defined as $(TP+TN)/(TP+TN+FP+FN)$. Although

a useful measure to consider, it is insufficient on its own as it is unable to differentiate between two possible mistakes made by the model – classifying an inactive listing as active (FP), and failing to classify an active listing (FN). Even though there is some kind of balance in the target classes, those two mistakes are not identical, and thus require additional evaluation.

Precision, $TP/(TP+FP)$, is the percentage of true positive predictions among all positive predictions. In the context of this research, Precision means the percentage of actually active listings among all listings predicted to be active. On the other hand, Recall is defined as $TP/(TP+FN)$, and represents the ratio of actual positive listings detected by the model. It is especially important when the cost of missing one active listing exceeds the cost of several false alarms. The F1-Score, $2 \cdot (Precision \cdot Recall) / (Precision + Recall)$, is the harmonic mean of the two aforementioned values and thus very useful when neither false positives nor false negatives should dominate the process.

6.2 Supervised model comparison

For performance measures, the Random Forest model exhibited the most successful results, including 0.860 for accuracy, 0.850 for precision, 0.891 for recall, 0.870 for F1, and 0.939 for ROC-AUC, which means that the model showed the highest effectiveness in distinguishing between active and inactive listings without generating many false positives. The Neural Network model worked well with almost similar metrics: accuracy - 0.845, precision - 0.825, F1 - 0.858, ROC-AUC – 0.925; however, this model had a slightly better recall (0.894). This implies that this model tried to be a bit more aggressive in predicting active listings, but at the same time, the model made more mistakes (Ulug & Akyüz (2025)). Gaussian Naïve Bayes was undoubtedly the fastest algorithm (training time - 0.181 seconds); however, the predictive performance of this model was very poor, including 0.679 for accuracy, 0.704 for F1, and 0.752 for ROC-AUC.

6.3 Clustering evaluation

Accuracy and F1 are not suitable measurements for evaluating K-means clustering, as there are no true class labels available. However, cluster quality was evaluated using inertia, elbow plot, silhouette score, and interpretability of the output cluster profile. Inertia is used to measure within-cluster cohesion, whereas an elbow plot provides a way to choose the number of clusters based on the decreasing gain in inertia for each increment of (K). Silhouette score, on the other

hand, is used to measure both cohesion and separation. The obtained silhouette score of 0.379 shows that the clusters are well-separated but not very well-separated – they have some use in segmentation and interpretation but are not highly separated.

6.4 Summary of modelling techniques

Modelling Technique	Purpose	Key Preprocessing	Evaluation Approach	Main Strength	Main Limitation
Random Forest	Predict recent review activity	Imputation and one-hot encoding	Accuracy, precision, recall, F1, ROC-AUC	Strong performance and nonlinear modelling	Less transparent than a single tree
Gaussian Naive Bayes	Probabilistic classification baseline	Imputation, encoding and scaling	Accuracy, precision, recall, F1, ROC-AUC	Extremely fast and simple	Independence and Gaussian assumptions
Neural Network	Nonlinear binary classification	Imputation, encoding and scaling	Classification metrics and learning curves	Flexible and high recall	Slow and difficult to interpret
K-means	Discover listing profiles	Imputation, outlier capping and scaling	Inertia, silhouette and cluster profiles	Reveals interpretable segments	Sensitive to scaling, outliers and K

6.5 Broader comparison

Of all the methods used in the project, the Random Forest needed moderate data pre-processing, and had a good ability to cope with non-linear dependencies without any distribution assumptions. This method was more robust than the Gaussian Naïve Bayes because the former doesn't depend on the assumption of conditional independence and Gaussian feature distributions, which cannot be completely true for this particular dataset (Kumar & Kotler (2024)). The neural network was the most costly in computing terms and was less understandable among others, but it was quite flexible and had a slightly better recall.

7. Reflection and Conclusions

In this stage of the project, I was required to make some modelling decisions that would influence the analysis. First of all, the lack of `price` forced me to redefine the target variable in terms of the observable outcome – `active\_listing`, which is equal to one if `number\_of\_reviews\_ltm > 0`. This variable was chosen because of its observability and defensibility, unlike the arbitrary definition of `review threshold`, which would require some reasoning for its introduction.

Some assumptions made during the modelling are as follows. I assume that recent reviews are a good proxy for recent activity since completed stays generally produce reviews, although not all of them do. Additionally, I believe that missing reviews can be considered as informative data since their absence can indicate listing inactivity. Finally, I assume that historical review count can be used despite the high collinearity between it and the target since we aim at prediction performance. Regarding clustering, I also assume that scaled numeric listing profiles can be clustered meaningfully via K-means.

In terms of data limitations, there are a few. First of all, the lack of the `price` variable means that value, revenue and pricing strategy cannot be analysed. Besides, the dataset lacks measures for booking performance, listing quality, amenities and temporal booking history. Therefore, my models provide explanations of review-related activity and not booking performance. I also faced imbalanced categories of room types as well as high-cardinality features such as name and license value, which were inappropriate for inclusion in the models.

There are also a few limitations associated with each modelling method. First, Random Forest shows good performance but provides non-causal impurity-based feature importances and remains semi-interpretable. Next, Naive Bayes is quick to compute but assumes unrealistic independence among correlated and skewed features. Third, a neural network captures non-linear dependencies but requires more computations and architectural design decisions to be made. Finally, K-means allows for segmentation but assumes spherical clusters, is very dependent on scaling and creates analytically constructed clusters rather than real-world groups.

Therefore, the above comparison allowed me to gain deeper insights into the process of model selection. In particular, simple models can be computationally efficient but underfit structured

datasets. Meanwhile, non-linear models handle this dataset better. However, it becomes evident that increased complexity does not guarantee higher performance (Chen et al., (2025)). Based on my findings, I would recommend Random Forest since it gives the best balance of accuracy, precision, F1, ROC-AUC, speed and interpretability. A neural network can remain preferable if recall is the primary focus.

Overall, I managed to find out that the recent review activity can be predicted quite accurately based on historical reviews, availability, minimum-stay requirement, host scale and location. Random Forest gave me the highest performance and interpretability along with computational efficiency, whereas clustering added meaningful insights. However, the results should be seen as patterns in review activity and not the actual booking performance.

Reference:

- Alibakhshi, A., & Hassannayebi, E. (2025). Towards an enhanced next activity prediction using attention based neural networks. *Cluster Computing*, 28(1), 54.
- Alves, C. J. (2024). *Airbnb Price Prediction in Lisbon and Porto: A Machine Learning Approach with Multi-Feature Integration* (Master's thesis, Universidade NOVA de Lisboa (Portugal)).
- Amarasinghe Arachchige, J. A. (2023). New perspectives on the real estate market based on machine learning and artificial intelligence approaches.
- Ayalasomayajula, M. (2024). Real-World Applications of Data Analytics, Big Data, and Machine. *Data Analytics and Machine Learning: Navigating the Big Data Landscape*, 237.
- Bai, Y., Zhang, Q., & Zheng, B. (2025). News-driven carbon trading market: machine learning-based carbon price prediction. *Environment, Development and Sustainability*, 1-41.
- Camatti, N., Di Tollo, G., Filograsso, G., & Ghilardi, S. (2024). Predicting Airbnb pricing: a comparative analysis of artificial intelligence and traditional approaches. *Computational Management Science*, 21(1), 30.
- Chaudhary, P. S., Khurana, M. R., & Ayalasomayajula, M. (2024). Real-world applications of data analytics, big data, and machine learning. In *Data Analytics and Machine Learning: Navigating the Big Data Landscape* (pp. 237-263). Singapore: Springer Nature Singapore.
- Chen, C., Huang, Y., Wu, S., Zhao, Y., & Xu, L. (2025). What makes you entrepreneurial? Using machine learning to predict technology entrepreneurship. *Baltic Journal of Management*, 20(4), 421-438.
- Costa, J. R. G. A. (2025). *Predicting Airbnb prices using machine learning algorithms* (Doctoral dissertation, Instituto Superior de Economia e Gestão).
- De Mauro, A., Sestino, A., & Bacconi, A. (2022). Machine learning and artificial intelligence use in marketing: a general taxonomy. *Italian Journal of Marketing*, 2022(4), 439-457.
- Gao Jr, Y. (2025). Unravelling the What, How, and When of Airbnb Price Prediction: A Case Study of London Based on Machine Learning.
- Ilieva, I., Terziyska, M., & Dimitrova, T. (2025). From Words to Ratings: Machine Learning and NLP for Wine Reviews. *Beverages*, 11(3), 80.

Kretschmer, M., Margoni, T., & Oruç, P. (2024). Copyright Law and the Lifecycle of Machine Learning Models: M. Kretschmer et al. *IIC-International Review of Intellectual Property and Competition Law*, 55(1), 110-138.

Kumar, V., & Kotler, P. (2024). Transformative marketing with machine learning (ML). In *Transformative marketing: Combining new age technologies and human insights* (pp. 103-140). Cham: Springer Nature Switzerland.

Kunze, A., La Torre, D., & Rocca, M. (2026). A bibliometric review of interpretability and explainability methods for artificial intelligence in economics and business. *Discover Artificial Intelligence*.

Mittal, D., Raj, V. E., & Kumar, V. (2023, August). Revolutionizing Finance and Travel with AI: Impacting through Machine Learning Models. In *Proceedings of the 2023 Fifteenth International Conference on Contemporary Computing* (pp. 274-281).

Mora-Garcia, R. T., Cespedes-Lopez, M. F., & Perez-Sanchez, V. R. (2022). Housing price prediction using machine learning algorithms in COVID-19 times. *Land*, 11(11), 2100.

Motamedi, G., Sheikh, A., Hashemi Golpayegani, A., & Khodabandehlou, S. (2026). A hybrid approach for customer churn prediction and prevention in tourism. *Information Technology & Tourism*, 28(1), 14.

Puri, V., Mondal, S., Das, S., & Vrana, V. G. (2023, January). Blockchain propels tourism industry—an attempt to explore topics and information in smart tourism management through text mining and machine learning. In *Informatics* (Vol. 10, No. 1, p. 9). MDPI.

Romanillos, G., & Moya-Gómez, B. (2023). New data and computational methods opportunities to enhance the knowledge base of tourism. In *Handbook of computational social science for policy* (pp. 361-379). Cham: Springer International Publishing.

Samsudeen, S., Ali, M. H., Vignesh, C. C., Kamruzzaman, M. M., Prakash, C., Thirugnanam, T., & Daniel, J. A. (2023). Context-specific discussion of Airbnb usage knowledge graphs for improving private social systems. *Journal of Combinatorial Optimization*, 45(2), 66.

Sharma, A. K., & Chiu, K. C. (2026). Data-driven prediction of borrower default in P2P lending using feature-optimized ML models. *OPSEARCH*, 1-24.

Sirigiri, P. (2024). IDENTIFICATION OF DISTRESSED REAL ESTATE PROPERTIES USING NATURAL LANGUAGE PROCESSING/MACHINE LEARNING.

Ulug, B., & Akyüz, S. (2025). Optimized churn prediction using ensemble-based feature selection via second-order cone programming. *Annals of Operations Research*, 353(2), 635-665.

Vasudevan, A. (2025). Airbnb Pricing Prediction Using Machine Learning: A Case Study on Seattle Listings.

Wang, D., & Zhang, Y. (2024). RETRACTED ARTICLE: Coupling of SME innovation and innovation in regional economic prosperity with machine learning and IoT technologies using XGBoost algorithm: D. Wang, Y. Zhang. *Soft Computing*, 28(4), 2919-2939.

Wyatt, K. (2023). *Airbnb Valuation: A Machine Learning Approach*. University of Arkansas.

Xishihui, D., Huifeng, S., Zhaoguo, W., Lishuang, S., & Qianqian, S. (2024). Investigating the determinants of homestay satisfaction on Airbnb using multiple techniques. *Scientific Reports*, 14(1), 24146.

Zang, Z., Fu, H., Cheng, J., Raza, H., & Fang, D. (2025). Digital threads of architectural heritage: navigating tourism destination image through social media reviews and machine learning insights. *Journal of Asian Architecture and Building Engineering*, 24(5), 4452-4469.

Appendix

```
!pip -q install scikit-learn tensorflow

import time
import random
import warnings
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
from sklearn.utils.class_weight import compute_class_weight
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score,
    classification_report,
    ConfusionMatrixDisplay,
    RocCurveDisplay,
    silhouette_score,
)

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers, callbacks

warnings.filterwarnings("ignore")

RANDOM_STATE = 42
np.random.seed(RANDOM_STATE)
random.seed(RANDOM_STATE)
tf.random.set_seed(RANDOM_STATE)

pd.set_option("display.max_columns", 100)
pd.set_option("display.float_format", lambda x: f"{x:,.3f}")
```

```
print("TensorFlow version:", tf.__version__)

DATASET_PATH = "/content/listings.csv"

if not Path(DATASET_PATH).exists():
    raise FileNotFoundError(
        f"Dataset not found at: {DATASET_PATH}\n"
        "Upload the file to Colab or update DATASET_PATH."
    )

df = pd.read_csv(DATASET_PATH)

print(f"Dataset shape: {df.shape[0]:,} rows and {df.shape[1]} columns")
display(df.head())

audit_table = pd.DataFrame(
    {
        "data_type": df.dtypes.astype(str),
        "missing_count": df.isna().sum(),
        "missing_percentage": df.isna().mean() * 100,
        "unique_values": df.nunique(dropna=False),
    }
).sort_values("missing_percentage", ascending=False)

display(audit_table)

print("Duplicate rows:", df.duplicated().sum())
if "id" in df.columns:
    print("Duplicate listing IDs:", df["id"].duplicated().sum())

all_missing_columns = [column for column in df.columns if
df[column].isna().all()]
print("Entirely missing columns:", all_missing_columns)

df_clean = df.drop(columns=all_missing_columns).copy()

if "last_review" in df_clean.columns:
    df_clean["last_review"] = pd.to_datetime(df_clean["last_review"],
errors="coerce")

if "number_of_reviews_ltm" not in df_clean.columns:
    raise KeyError("Required column missing: number_of_reviews_ltm")

df_clean["active_listing"] = (df_clean["number_of_reviews_ltm"] >
0).astype(int)

if "license" in df_clean.columns:
    df_clean["has_license"] = df_clean["license"].notna().astype(int)
```

```

else:
    df_clean["has_license"] = 0

target_summary = (
    df_clean["active_listing"]
    .value_counts()
    .sort_index()
    .rename(index={0: "Inactive", 1: "Active"})
    .to_frame("count")
)
target_summary["percentage"] = target_summary["count"] / len(df_clean)
* 100

display(target_summary)

numeric_columns = df_clean.select_dtypes(include=np.number).columns
display(df_clean[numeric_columns].describe().T)

if "room_type" in df_clean.columns:
    print("Room types")
    display(df_clean["room_type"].value_counts().to_frame("count"))

if "neighbourhood" in df_clean.columns:
    print("Neighbourhoods")
    display(df_clean["neighbourhood"].value_counts().to_frame("count"))

missing_percentage =
df_clean.isna().mean().mul(100).sort_values(ascending=False)
missing_percentage = missing_percentage[missing_percentage > 0]

if not missing_percentage.empty:
    plt.figure(figsize=(10, 5))
    missing_percentage.plot(kind="bar")
    plt.title("Percentage of Missing Values by Variable")
    plt.xlabel("Variable")
    plt.ylabel("Missing values (%)")
    plt.xticks(rotation=45, ha="right")
    plt.tight_layout()
    plt.show()

plt.figure(figsize=(6, 4))
(
    df_clean["active_listing"]
    .map({0: "Inactive", 1: "Active"})
    .value_counts()
    .plot(kind="bar")
)
plt.title("Distribution of the Supervised Target")

```

```
plt.xlabel("Listing status")
plt.ylabel("Number of listings")
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()

if "room_type" in df_clean.columns:
    plt.figure(figsize=(8, 5))
    df_clean["room_type"].value_counts().plot(kind="bar")
    plt.title("Distribution of Room Types")
    plt.xlabel("Room type")
    plt.ylabel("Number of listings")
    plt.xticks(rotation=25, ha="right")
    plt.tight_layout()
    plt.show()

    room_activity = pd.crosstab(
        df_clean["room_type"],
        df_clean["active_listing"],
        normalize="index",
    ).mul(100)

    room_activity.columns = ["Inactive (%)", "Active (%)"]
    display(room_activity)

if "neighbourhood" in df_clean.columns:
    neighbourhood_activity = (
        df_clean.groupby("neighbourhood")["active_listing"]
        .agg(active_rate="mean", listing_count="size")
        .sort_values("active_rate")
    )

    plt.figure(figsize=(9, 7))
    neighbourhood_activity["active_rate"].mul(100).plot(kind="barh")
    plt.title("Percentage of Active Listings by Neighbourhood")
    plt.xlabel("Active listings (%)")
    plt.ylabel("Neighbourhood")
    plt.tight_layout()
    plt.show()

    display(neighbourhood_activity)

eda_numeric_columns = [
    "minimum_nights",
    "number_of_reviews",
    "calculated_host_listings_count",
    "availability_365",
]
```

```

for column in eda_numeric_columns:
    if column not in df_clean.columns:
        continue

    values = df_clean[column].dropna()
    upper_limit = values.quantile(0.99)
    displayed_values = values[values <= upper_limit]

    plt.figure(figsize=(8, 4))
    plt.hist(displayed_values, bins=40)
    plt.title(f"Distribution of {column} (up to 99th percentile)")
    plt.xlabel(column)
    plt.ylabel("Frequency")
    plt.tight_layout()
    plt.show()

feature_columns = [
    "neighbourhood",
    "room_type",
    "latitude",
    "longitude",
    "minimum_nights",
    "number_of_reviews",
    "calculated_host_listings_count",
    "availability_365",
]

categorical_features = ["neighbourhood", "room_type"]
numeric_features = [
    "latitude",
    "longitude",
    "minimum_nights",
    "number_of_reviews",
    "calculated_host_listings_count",
    "availability_365",
]

missing_required = [col for col in feature_columns if col not in
df_clean.columns]
if missing_required:
    raise KeyError(f"Missing required feature columns:
{missing_required}")

X = df_clean[feature_columns].copy()
y = df_clean["active_listing"].copy()

X_train, X_test, y_train, y_test = train_test_split(

```

```

X,
y,
test_size=0.20,
random_state=RANDOM_STATE,
stratify=y,
)

print("Training shape:", X_train.shape)
print("Test shape:", X_test.shape)
display(y_train.value_counts(normalize=True).sort_index())
tree_preprocessor = ColumnTransformer(
    transformers=[
        (
            "numeric",
            Pipeline(
                steps=[
                    ("imputer", SimpleImputer(strategy="median")),
                ]
            ),
            numeric_features,
        ),
        (
            "categorical",
            Pipeline(
                steps=[
                    ("imputer",
SimpleImputer(strategy="most_frequent")),
                    (
                        "onehot",
                        OneHotEncoder(handle_unknown="ignore",
sparse_output=False),
                    ),
                ]
            ),
            categorical_features,
        ),
    ]
)

scaled_preprocessor = ColumnTransformer(
    transformers=[
        (
            "numeric",
            Pipeline(
                steps=[
                    ("imputer", SimpleImputer(strategy="median")),
                    ("scaler", StandardScaler()),
                ]
            ),
            numeric_features,
        ),
        (
            "categorical",
            Pipeline(
                steps=[
                    ("imputer",
SimpleImputer(strategy="most_frequent")),
                    (
                        "onehot",
                        OneHotEncoder(handle_unknown="ignore",
sparse_output=False),
                    ),
                ]
            ),
            categorical_features,
        ),
    ]
)

```

```

        ),
        numeric_features,
    ),
    (
        "categorical",
        Pipeline(
            steps=[
                ("imputer",
SimpleImputer(strategy="most_frequent")),
                (
                    "onehot",
                    OneHotEncoder(handle_unknown="ignore",
sparse_output=False),
                ),
            ]
        ),
        categorical_features,
    ),
]
)

model_results = []

def evaluate_classifier(model_name, y_true, predictions, probabilities,
fit_time):
    result = {
        "Model": model_name,
        "Accuracy": accuracy_score(y_true, predictions),
        "Precision": precision_score(y_true, predictions,
zero_division=0),
        "Recall": recall_score(y_true, predictions, zero_division=0),
        "F1": f1_score(y_true, predictions, zero_division=0),
        "ROC_AUC": roc_auc_score(y_true, probabilities),
        "Fit_time_seconds": fit_time,
    }

    model_results.append(result)
    display(pd.DataFrame([result]))

print(
    classification_report(
        y_true,
        predictions,
        target_names=["Inactive", "Active"],
        zero_division=0,
    )
)

```



```

ConfusionMatrixDisplay.from_predictions(
    y_true,
    predictions,
    display_labels=["Inactive", "Active"],
)
plt.title(f"{model_name}: Confusion Matrix")
plt.tight_layout()
plt.show()

RocCurveDisplay.from_predictions(
    y_true,
    probabilities,
    name=model_name,
)
plt.title(f"{model_name}: ROC Curve")
plt.tight_layout()
plt.show()

random_forest = Pipeline(
    steps=[
        ("preprocessor", tree_preprocessor),
        (
            "classifier",
            RandomForestClassifier(
                n_estimators=250,
                max_depth=16,
                min_samples_leaf=5,
                class_weight="balanced",
                random_state=RANDOM_STATE,
                n_jobs=-1,
            ),
        ),
    ],
)

start_time = time.perf_counter()
random_forest.fit(X_train, y_train)
rf_fit_time = time.perf_counter() - start_time

rf_predictions = random_forest.predict(X_test)
rf_probabilities = random_forest.predict_proba(X_test)[:, 1]

evaluate_classifier(
    "Random Forest",
    y_test,
    rf_predictions,
    rf_probabilities,
    rf_fit_time,

```

```

)

rf_preprocessor = random_forest.named_steps["preprocessor"]
rf_classifier = random_forest.named_steps["classifier"]

feature_names = rf_preprocessor.get_feature_names_out()
feature_importance = pd.Series(
    rf_classifier.feature_importances_,
    index=feature_names,
).sort_values(ascending=False)

display(feature_importance.head(20).to_frame("importance"))

plt.figure(figsize=(9, 7))
feature_importance.head(15).sort_values().plot(kind="barh")
plt.title("Random Forest: Top 15 Feature Importances")
plt.xlabel("Mean decrease in impurity")
plt.ylabel("Feature")
plt.tight_layout()
plt.show()

gaussian_nb = Pipeline(
    steps=[
        ("preprocessor", scaled_preprocessor),
        ("classifier", GaussianNB()),
    ]
)

start_time = time.perf_counter()
gaussian_nb.fit(X_train, y_train)
nb_fit_time = time.perf_counter() - start_time

nb_predictions = gaussian_nb.predict(X_test)
nb_probabilities = gaussian_nb.predict_proba(X_test)[:, 1]

evaluate_classifier(
    "Gaussian Naive Bayes",
    y_test,
    nb_predictions,
    nb_probabilities,
    nb_fit_time,
)

X_train_nn = scaled_preprocessor.fit_transform(X_train)
X_test_nn = scaled_preprocessor.transform(X_test)

X_train_nn = np.asarray(X_train_nn).astype("float32")
X_test_nn = np.asarray(X_test_nn).astype("float32")

```

```

y_train_nn = y_train.to_numpy().astype("float32")
y_test_nn = y_test.to_numpy().astype("float32")

classes = np.unique(y_train)
class_weights_array = compute_class_weight(
    class_weight="balanced",
    classes=classes,
    y=y_train,
)
class_weight_dictionary = {
    int(class_label): float(weight)
    for class_label, weight in zip(classes, class_weights_array)
}

print("Neural-network input shape:", X_train_nn.shape)
print("Class weights:", class_weight_dictionary)

tf.keras.backend.clear_session()

neural_network = keras.Sequential(
    [
        layers.Input(shape=(X_train_nn.shape[1],)),
        layers.Dense(32, activation="relu"),
        layers.Dense(16, activation="relu"),
        layers.Dense(1, activation="sigmoid"),
    ]
)

neural_network.compile(
    optimizer=keras.optimizers.Adam(learning_rate=0.001),
    loss="binary_crossentropy",
    metrics=[
        keras.metrics.BinaryAccuracy(name="accuracy"),
        keras.metrics.Precision(name="precision"),
        keras.metrics.Recall(name="recall"),
        keras.metrics.AUC(name="auc"),
    ],
)

neural_network.summary()

early_stopping = callbacks.EarlyStopping(
    monitor="val_loss",
    patience=8,
    restore_best_weights=True,
)

```

```

start_time = time.perf_counter()
history = neural_network.fit(
    X_train_nn,
    y_train_nn,
    validation_split=0.15,
    epochs=100,
    batch_size=256,
    class_weight=class_weight_dictionary,
    callbacks=[early_stopping],
    verbose=1,
)
nn_fit_time = time.perf_counter() - start_time
history_table = pd.DataFrame(history.history)

plt.figure(figsize=(8, 5))
plt.plot(history_table["loss"], label="Training loss")
plt.plot(history_table["val_loss"], label="Validation loss")
plt.title("Neural Network Loss")
plt.xlabel("Epoch")
plt.ylabel("Binary cross-entropy")
plt.legend()
plt.tight_layout()
plt.show()

plt.figure(figsize=(8, 5))
plt.plot(history_table["accuracy"], label="Training accuracy")
plt.plot(history_table["val_accuracy"], label="Validation accuracy")
plt.title("Neural Network Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.tight_layout()
plt.show()

nn_probabilities = neural_network.predict(X_test_nn, verbose=0).ravel()
nn_predictions = (nn_probabilities >= 0.5).astype(int)

evaluate_classifier(
    "Feed-forward Neural Network",
    y_test,
    nn_predictions,
    nn_probabilities,
    nn_fit_time,
)

results_df = (
    pd.DataFrame(model_results)
    .sort_values("F1", ascending=False)

```

```

        .reset_index(drop=True)
    )

display(results_df)

results_df.set_index("Model")[["Accuracy", "Precision", "Recall", "F1",
"ROC_AUC"]].plot(
    kind="bar",
    figsize=(10, 5),
)
plt.ylim(0, 1)
plt.title("Supervised Model Comparison")
plt.xlabel("Model")
plt.ylabel("Score")
plt.xticks(rotation=20, ha="right")
plt.tight_layout()
plt.show()

cluster_features = [
    "latitude",
    "longitude",
    "minimum_nights",
    "number_of_reviews",
    "calculated_host_listings_count",
    "availability_365",
]

missing_cluster_features = [col for col in cluster_features if col not
in df_clean.columns]
if missing_cluster_features:
    raise KeyError(f"Missing required clustering columns:
{missing_cluster_features}")

cluster_data = df_clean[cluster_features].copy()

for column in [
    "minimum_nights",
    "number_of_reviews",
    "calculated_host_listings_count",
]:
    upper_cap = cluster_data[column].quantile(0.99)
    cluster_data[column] = cluster_data[column].clip(upper=upper_cap)

cluster_preprocessor = Pipeline(
    steps=[
        ("imputer", SimpleImputer(strategy="median")),
        ("scaler", StandardScaler()),
    ]

```

```
)

cluster_matrix = cluster_preprocessor.fit_transform(cluster_data)
print("Clustering matrix shape:", cluster_matrix.shape)

k_values = range(2, 9)
inertias = []
silhouette_scores = []

rng = np.random.default_rng(RANDOM_STATE)
sample_size = min(10000, len(cluster_matrix))
sample_indices = rng.choice(len(cluster_matrix), size=sample_size,
                             replace=False)
cluster_sample = cluster_matrix[sample_indices]

for k in k_values:
    candidate_model = KMeans(
        n_clusters=k,
        n_init=20,
        random_state=RANDOM_STATE,
    )
    candidate_labels = candidate_model.fit_predict(cluster_matrix)
    inertias.append(candidate_model.inertia_)
    silhouette_scores.append(
        silhouette_score(cluster_sample,
                           candidate_labels[sample_indices])
    )

selection_table = pd.DataFrame(
    {
        "k": list(k_values),
        "inertia": inertias,
        "silhouette_score": silhouette_scores,
    }
)

display(selection_table)

plt.figure(figsize=(7, 4))
plt.plot(selection_table["k"], selection_table["inertia"], marker="o")
plt.title("K-means Elbow Plot")
plt.xlabel("Number of clusters (K)")
plt.ylabel("Inertia")
plt.tight_layout()
plt.show()

plt.figure(figsize=(7, 4))
```

```
plt.plot(selection_table["k"], selection_table["silhouette_score"],
marker="o")
plt.title("K-means Silhouette Scores")
plt.xlabel("Number of clusters (K)")
plt.ylabel("Silhouette score")
plt.tight_layout()
plt.show()

best_k =
int(selection_table.loc[selection_table["silhouette_score"].idxmax(),
"k"])

final_kmeans = KMeans(
    n_clusters=best_k,
    n_init=30,
    random_state=RANDOM_STATE,
)

df_clean["cluster"] = final_kmeans.fit_predict(cluster_matrix)

print("Selected number of clusters:", best_k)

display(
    df_clean["cluster"]
    .value_counts()
    .sort_index()
    .to_frame("listing_count")
)

cluster_profile = (
    df_clean.groupby("cluster")[cluster_features]
    .median()
    .round(2)
)

cluster_profile["listing_count"] = df_clean.groupby("cluster").size()
cluster_profile["active_listing_percentage"] = (
    df_clean.groupby("cluster")["active_listing"]
    .mean()
    .mul(100)
    .round(2)
)

display(cluster_profile)

if "room_type" in df_clean.columns:
    room_type_profile = pd.crosstab(
        df_clean["cluster"],
```

```

        df_clean["room_type"],
        normalize="index",
    ).mul(100).round(2)
    display(room_type_profile)
else:
    room_type_profile = pd.DataFrame()

pca = PCA(n_components=2, random_state=RANDOM_STATE)
cluster_pca = pca.fit_transform(cluster_matrix)

plot_sample_size = min(12000, len(df_clean))
plot_indices = rng.choice(len(df_clean), size=plot_sample_size,
replace=False)
sample_labels = df_clean.iloc[plot_indices]["cluster"].to_numpy()

plt.figure(figsize=(9, 7))
for cluster_id in sorted(df_clean["cluster"].unique()):
    mask = sample_labels == cluster_id
    plt.scatter(
        cluster_pca[plot_indices][mask, 0],
        cluster_pca[plot_indices][mask, 1],
        s=10,
        alpha=0.45,
        label=f"Cluster {cluster_id}",
    )

plt.title("K-means Clusters Projected onto Two Principal Components")
plt.xlabel(f"PC1 ({pca.explained_variance_ratio_[0] * 100:.1f}% variance)")
plt.ylabel(f"PC2 ({pca.explained_variance_ratio_[1] * 100:.1f}% variance)")
plt.legend()
plt.tight_layout()
plt.show()

print(
    "Variance explained by PC1 and PC2:",
    f"{pca.explained_variance_ratio_.sum() * 100:.2f}%",
)

results_df.to_csv("supervised_model_comparison.csv", index=False)
feature_importance.to_csv("random_forest_feature_importance.csv",
header=["importance"])
selection_table.to_csv("kmeans_selection_metrics.csv", index=False)
cluster_profile.to_csv("cluster_profiles.csv")

if not room_type_profile.empty:
    room_type_profile.to_csv("cluster_room_type_profile.csv")

```



```
print("All result tables were saved successfully.")
```